Harjoitusprojekti #4

Lotto & Pelihistoria (Random, Files, Exceptions)

Tavoite

Rakennat C#-konsoliohjelman, joka:

1. arpoo **lotto-rivejä** satunnaisuudella (Random)
2. validoi syötteitä ja heittää tarvittaessa **ArgumentException**
3. tallentaa ja lataa **pelihistorian tiedostosta** (File/StreamWriter) ja käsittelee tiedostovirheitä try/catchilla

Käytettävät syntaksit/komennot (pakolliset)

- `Random + Next(...) / Next(min,max)`
- `List<int> + Add(...) + Contains(...) + foreach (lottonumerot)`
- `try { } catch (Exception e) { } tiedostojen luku/kirjoitus`
- `throw new ArgumentException(...) parametrien validointiin`
- `File.ReadAllLines(...) / File.ReadAllText(...) ja StreamWriter / File.WriteAllText(...)`

Projektin idea (lyhyesti)

Ohjelma arpoo lotto-rivin (7 eri numeroa väliltä 1–40), vertailee sitä käyttäjän “omaa riviä” vastaan, ja tallentaa jokaisen pelin tuloksen **pelihistoriaan tiedostoon**.

Palautettavat tiedostot

- `Program.cs`
- `LotteryRow.cs`
- `Player.cs`
- `GameHistory.cs`
- `history.txt` (automaattisesti syntyvä tiedosto)

Osa A — Satunnaisuus: Lotto-rivi (LotteryRow)

Tee luokka `LotteryRow`

Kenttä:

- `private List<int> numbers;`

Metodit:

- `public LotteryRow()` → arpoo numerot kutsumalla `RandomizeNumbers()`
- `public List<int> Numbers()` → palauttaa listan
- `public bool ContainsNumber(int number)` → kertoo löytyykö numero
- `public void RandomizeNumbers()` → arpoo 7 **eri** numeroa väliltä 1–40

Hyväksymiskriteerit

- rivissä on aina **7 kpl** numeroita
- kaikki numerot ovat välillä **1..40**
- samassa rivissä ei ole duplikaatteja

Osa B — Poikkeukset: Validointi (ArgumentException)

Tee luokka `Player`

Kentät:

- `public string Name { get; }`
- `public LotteryRow MyRow { get; }`

Konstruktori:

- `public Player(string name, LotteryRow myRow)`

Validointi (pakollinen):

- jos `name` on `null`, tyhjä tai yli 40 merkkiä → `throw new ArgumentException("...")`
- jos `myRow` on `null` → `throw new ArgumentException("...")`

Hyväksymiskriteerit

- virheellisellä nimellä ohjelma ei kaadu hallitsemattomasti, vaan poikkeus on järkevä ja kertoo syyn
- validilla nimellä Player luodaan onnistuneesti

Osa C — Tiedostot + try/catch: Pelihistoria

Tee luokka `GameHistory`

Kentät:

- `private string fileName;`
- `private List<string> lines;`

Konstruktori:

- `public GameHistory(string fileName)` → tallentaa tiedoston nimen ja alustaa listan

Metodit:

1. `public bool Load()`
   - yrittää lukea tiedoston: `File.ReadAllLines(fileName)`
   - jos onnistuu → talleta rivit `lines`-listaan ja palauta `true`
   - jos epäonnistuu → `catch (Exception e)` tulosta viesti ja palauta `false`
2. `public bool Save()`
   - kirjoittaa `lines` tiedostoon (`StreamWriter` tai `File.WriteAllText`) try/catchilla
   - palauttaa `true` jos onnistui, muuten `false`
3. `public void AddLine(string line)`
   - lisää rivin historiaan (muistiin)
4. `public void Print()`
   - tulostaa historian rivit `foreach`-loopilla

Hyväksymiskriteerit

- jos tiedostoa ei ole, `Load()` ei kaada ohjelmaa (palauttaa false + tulostaa virheen)
- `Save()` luo/ylikirjoittaa tiedoston ja tallentaa rivit

Osa D — Pääohjelma: peli + tallennus

Toteuta pelilogiikka Program.cs:ään

Flow:

1. Käynnistyessä:
   - luo `GameHistory("history.txt")`
   - kutsu `Load()` (jos false, jatka silti tyhjällä historialla)
2. Luo pelaaja:
   - kysy nimi (tai käytä valmista arvoa)
   - luo `Player(name, myRow)`
   - ympäröi pelaajan luonti `try/catch (ArgumentException e)` → tulosta viesti
3. Arvo "voittorivi":
   - `LotteryRow winningRow = new LotteryRow();`
4. Laske osumat:
   - käy pelaajan numerot läpi ja tarkista `winningRow.ContainsNumber(...)`
5. Tallenna tulos historiaan yhtenä rivinä, esim:
   - `2026-02-18; Name=Anna; Hits=3; My=...; Win=...`
   - lisää se `AddLine(...)`
   - lopuksi `Save()`
6. Tulosta historia lopuksi

Hyväksymiskriteerit

- ohjelma arpoo rivit joka ajolla eri tavalla (satunnaisuus)
- virheellinen nimi ei kaada ohjelmaa (ArgumentException käsitellään)
- historia jää talteen `history.txt`-tiedostoon ja latautuu seuraavalla ajolla